

DA5402W - Machine Learning Operations Lab

RouteSense: MLOps-Driven Predictive Analytics for Logistics Delivery Performance

Submitted by - Team 45

Akshaya Sanal : DA25G503
Anoop Sudevan : DA25M547
Devika Dileep : DA25M561
Vivek Vijayan : DA25M638

August 25, 2026

Abstract

Traditional logistics navigation engines fails to evaluate physical vehicle stresses and dynamic operational constraints under heavy-duty enterprise transit conditions. This report introduces RouteSense, an end-to-end automated Machine Learning Operations (MLOps) system designed to solve complex multi-track logistics targets simultaneously without introducing data leakage.

Utilizing a 1,000-row e-commerce logistics dataset containing 18 delivery metrics, the architecture decouples a shared data pool into four independent feature tracks: Geography, Environment, Telemetry/Cargo, and Dynamic Operational Cost. The automated system infrastructure factory streams raw parameters row-by-row via an Apache Kafka broker, schedules batch job dependencies through Apache Airflow DAG workflows, and executes distributed feature cleaning using an Apache Spark processing engine.

Data Version Control (DVC) locks and versions dataset splits for full pipeline reproducibility, while an MLflow Registry tracks randomized hyperparameter optimization search sweeps and secures frozen model files. For live deployment, the main model binaries are loaded directly into memory inside a unified FastAPI web gateway router packaged within cloud-native Docker container services. The serving engine processes incoming user form payloads against strict type contracts, executing parallel inference loops across all four model architectures simultaneously to output a comprehensive fleet performance scorecard.

Contents

1	Introduction	5
1.1	Project Statement	5
1.2	Workload Distribution Strategy	5
2	System Architecture	7
2.1	Pipeline Workflow Topology	7
2.2	Data Management & Ingestion Components	7
2.3	Strict Feature Masking Matrix	7
2.4	Model Tracking, Serving, & Automation	8
3	Model Implementation	9
3.1	Two-Way Training and Model Comparison Strategy	9
3.2	Track 1 — Spatial Geography Geography Pipeline Implementation	9
3.2.1	Distributed Feature Processing and Data Leakage Mitigation	9
3.2.2	Hyperparameter Optimization and Production Registration	10
3.3	Track 2 — Operational Cost Pipeline Implementation	14
3.3.1	Distributed Feature Processing and Data Leakage Mitigation	14
3.3.2	Hyperparameter Optimization and Production Registration	14
3.4	Track 3 — Conditions and Route Reliability Pipeline Implementation	18
3.4.1	Feature Engineering and Target Formulation	18
3.4.2	Hyperparameter Optimization and Production Registration	18
3.5	Track 4 — Cargo Payload Pipeline Implementation	21
3.5.1	Distributed Feature Processing and Data Leakage Mitigation	21
3.5.2	Hyperparameter Optimization and Production Registration	21
4	System Integration	26
4.1	System Integration Overview	26
4.2	Integrated Output and Service Matrix	26
5	Conclusion and Future Scope	29

List of Tables

2.1	RouteSense Inbound Features and Multi-Track Target Mapping	8
3.1	Geography Track Production Performance Metrics Comparison	10
3.2	Cost Track Production Performance Metrics Comparison	14
3.3	Conditions Track Production Performance Metrics	18
3.4	Cargo Payload Track Production Performance Metrics Comparison	22
4.1	Integrated MLOps Ecosystem Service-Level Port Matrix	27

List of Figures

2.1	RouteSense Multi-Track End-to-End Core MLOps Infrastructure Flowchart . . .	7
3.1	MLflow tracking experiment dashboard comparing candidate model runs (Random Forest vs. Linear Regression).	10
3.2	MLflow model schema, input signatures, and the centralized Model Registry interface securing version 1 of the champion <code>geography_route_time_model</code>	11
3.3	Interactive FastAPI Swagger UI gateway interface validating incoming spatial feature payloads under Pydantic contracts.	12
3.4	Successful HTTP 200 asynchronous server response outputting the real-time optimized route travel duration estimation.	12
3.5	Evidently Data Drift dashboard highlighting feature distribution divergences and statistical test results across incoming production batches.	12
3.6	Apache Airflow DAG visual graph view displaying the sequential execution of containerized Spark preprocessing and training tasks.	13
3.7	Airflow task instance execution list tracking containerized DockerOperator run durations and success states.	13
3.8	MLflow tracking interface showing candidate evaluations under the Cost Track experiment.	15
3.9	Cost Production run dashboard displaying champion metrics (R^2 : 0.9185) and the registered <code>cost_route_cost_model_artifact_bundle</code>	16
3.10	Evidently Data Drift monitoring report confirming stable distribution bounds (11.11% drift share across 18 evaluated columns).	17
3.11	FastAPI operational serving gateway: (Top) Interactive Swagger UI input payload validation, (Bottom) Successful HTTP 200 JSON prediction response. . . .	17
3.12	MLflow evaluation dashboard displaying hyperparameters and performance metrics (R^2 : 0.9982) for the Final XGBoost conditions model.	19
3.13	MLflow Model Registry interface securing version 4 of the <code>conditions-route-reliability</code> champion artifact bundle.	19
3.14	Logistics Router API serving gateway for Track 3: (Top) Interactive Swagger UI payload input, (Bottom) Successful HTTP 200 JSON prediction response returning the computed route reliability index.	20
3.15	MLflow tracking experiment dashboard comparing candidate model runs (Random Forest vs. XGBoost).	22
3.16	MLflow Model Registry	23
3.17	Interactive FastAPI Swagger UI gateway interface validating incoming spatial feature payloads under Pydantic contracts	24
3.18	Successful HTTP 200 asynchronous server response outputting the real-time optimized route travel duration estimation.	24
3.19	Evidently Data Drift dashboard highlighting feature distribution divergences and statistical test results across incoming production batches.	25
4.1	End-to-End Multi-Track MLOps Integration Architecture Flow.	26

4.2	Apache Airflow Pipeline Orchestration	27
4.3	MLflow Model Registry	27
4.4	Integrated Logistics Router FastAPI Interface	28
4.5	Integrated API Prediction Output	28

Chapter 1

Introduction

1.1 Project Statement

When we use navigation apps like Google Maps on our phones, we only see the fastest or shortest path to our destination. This works great for general passenger cars, but commercial logistics companies face a different set of challenges. A standard app cannot tell how loading 40 tons of heavy cargo changes a truck’s axle strain, how running low tire pressure burns more fuel over hills, or how engine RPM anomalies flag a breakdown risk before the truck leaves the warehouse.

Our project, RouteSense, aims to fix this industrial gap. We built a machine learning operations (MLOps) platform that uses historical delivery logs from urban e-commerce operations. Instead of calculating just one path, our application reads a single input contract packet containing 14 real-world features and evaluates 4 distinct targets at the same time: dynamic delivery costs, trip durations, delivery efficiency scores, and route reliability indexes.

1.2 Workload Distribution Strategy

To guarantee full system ownership and ensure every team member understands the complete MLOps workflow, we split our data metrics into four separate project tracks. Each student fully containerized, tested, and deployed their own individual feature routes from scratch to establish a clean engineering division.

The primary reason for this four-track split is to enforce **Strict Feature Masking** right after ingestion, which completely eliminates the risk of **Data Leakage** across our four separate target variables. If a model runs its optimization loops over a shared dataset file containing alternative target columns, it can mathematically peek at those columns during training. This causes inflated, fake validation metrics in our local notebooks while causing the model to fail in production. By assigning an independent pipeline to each student, our individual data cleaning scripts explicitly drop all competing target columns. This ensures our models derive their coefficients purely from their assigned features, keeping the system clean and completely safe from cross-contamination.

Additionally, this distribution pattern ensures complete workload decoupling. Separating the data fields allows each team member to manage an independent script pipeline (`clean.py` → `tune.py` → `train.py` → `app.py`) on their own isolated compute environment. This allows individual tracks to run heavy randomized hyperparameter tuning loops or scale their ingestion configurations without draining server memory or bringing down the centralized FastAPI mesh router gateway. The final multi-track workload assignments are distributed as follows:

- **Track 1 — Spatial Geography (Devika Dileep, DA25M561):** Focuses on geographic coordinates and route lengths using variables like `distance_km`, `order_latitude`,

and `order_longitude`. This track isolates these spatial fields to train models predicting the estimated delivery duration target (`optimized_route_time_min`).

- **Track 2 — Operational Cost (Akshaya Sanal, DA25G503):** Focuses on logistics resource pricing, travel velocities, and hourly labor costs using parameters like `average_speed_kmph`, `fuel_cost_per_km`, and `driver_cost_per_hour`. This track models final route expenditures to predict the continuous cost target (`optimized_route_cost`).
- **Track 3 — Environmental Conditions (Vivek Vijeyan, DA25M638):** Focuses on traffic levels, delivery window allowances, and weather disruptions using metrics like `time_of_day`, `delivery_time_window_hrs`, `traffic_density_index`, and `weather_impact_index`. The goal here is to predict the route stability target under varying city conditions (`route_reliability_index`).
- **Track 4 — Cargo Payload (Anoop Sudevan, DA25M547):** Focuses on cargo weight rules and truck dimensions using variables like `order_weight_kg`, `cargo_weight_kg`, `vehicle_capacity_pct`, `delivery_deadline_hrs`, `trip_distance_km`, `traffic_distance`

Chapter 2

System Architecture

2.1 Pipeline Workflow Topology

The structural blueprint of the RouteSense platform follows a completely decoupled MLOps lifecycle. To illustrate the exact execution steps of our data engineering, modeling, and serving tasks, the global system topology is mapped sequentially in the flowchart below (Figure 2.1).

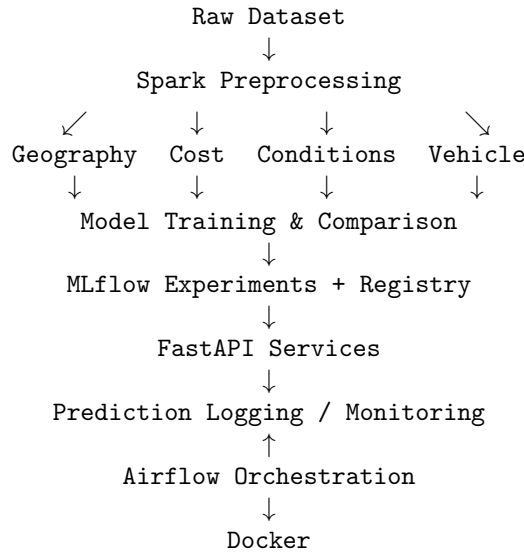


Figure 2.1: RouteSense Multi-Track End-to-End Core MLOps Infrastructure Flowchart

2.2 Data Management & Ingestion Components

Our backend infrastructure handles delivery records over a multi-stage production network. The raw logistics dataset is loaded directly into our **Spark Preprocessing** engine, where distributed data cleaning takes over. Spark processes our 1,000 baseline records by filtering out impossible outlier readings (such as negative speeds), dropping missing geographic coordinate lines, and filling structural feature gaps using column averages.

2.3 Strict Feature Masking Matrix

As shown in Figure 2.1, the single master dataset is partitioned inside Spark into four independent feature tracks using a strict feature masking design pattern to completely eliminate

the risk of data leakage. By assigning an independent pipeline branch to each student, our individual data cleaning scripts explicitly drop all competing target columns. This ensures that each model stays completely blind to alternative outcomes during training loops, deriving its coefficients purely from its assigned feature columns.

Table 2.1: RouteSense Inbound Features and Multi-Track Target Mapping

Track Name	Input Feature Variables	Target Out
1. Geography	distance_km, order_latitude, order_longitude	optimized_r
2. Cost Track	average_speed_kmph, fuel_cost_per_km, driver_cost_per_hour	optimized_r
3. Conditions	time_of_day, weather_impact_index, delivery_time_window_hrs, traffic_density_index	route_relia
4. Cargo Payload Track	cargo_weight_kg, vehicle, delivery_deadline_hrs, trip_distance_km traffic_density_index weather_impact_index overloaded_flag	engine_wear

2.4 Model Tracking, Serving, & Automation

Once the masked datasets are exported from Spark, they are pulled into our independent **Model Training & Comparison** loops. **MLflow Tracking** logs our randomized hyperparameter cross-validation runs, captures loss metrics, and saves our winning champion binaries into a central model registry.

For production serving, these registered binaries are loaded directly into memory inside our **FastAPI Services** router. The application uses an asynchronous single-input contract layout. When a client query hits the gateway, it runs parallel inference loops across all four models simultaneously, delivering a comprehensive performance scorecard. Every step of this live inference pipeline, alongside our performance tracking dashboards, is automatically run by **Airflow Orchestration** and packaged inside isolated **Docker** container environments to guarantee total portability across different machines.

Chapter 3

Model Implementation

3.1 Two-Way Training and Model Comparison Strategy

To discover the absolute highest predictive accuracy for each individual logistics target, our team implemented a synchronized two-way model training and evaluation strategy. Instead of blindly trusting a single machine learning algorithm, each track trained and compared two distinct algorithms side-by-side on their masked feature matrices. This rigorous comparison allows us to evaluate how simpler linear setups stack up against more complex, non-linear tree-based architectures under the same dataset constraints.

During the training phase, the isolated track datasets were split into 80 percent training arrays to fit the mathematical coefficients and 20 percent validation sets to test performance on unseen records. We evaluated our regression targets using standard industry metrics: Mean Absolute Error (MAE) to check linear errors, Mean Squared Error (MSE) or Root Mean Squared Error (RMSE) to measure average prediction mistakes, and the R-squared (R^2) score to track the percentage of data variance our models successfully captured.

3.2 Track 1 — Spatial Geography Geography Pipeline Implementation

The Spatial Geography track, managed by Devika Dileep (DA25M561), focuses on predicting the estimated journey travel duration target variable (`optimized_route_time_min`) using spatial logistics attributes. To enforce clean lifecycle boundaries and separate heavy compute requirements from continuous serving tasks, the operational pipeline enforces a strict decoupling architecture split across four separate production scripts: `spark_preprocessing.py` → `tune_hyperparameters.py` → `train_final_model.py` → `app.py`.

3.2.1 Distributed Feature Processing and Data Leakage Mitigation

The pipeline ingestion begins by building a container-native **PySpark** data session locally to handle the 1,000 baseline delivery instances. During initial exploratory analysis, a critical domain-informed data leakage check was executed. It was discovered that a derived route-time calculation combining distance, traffic indices, and `average_speed_kmph` yielded a near-perfect pairwise correlation of 0.999999999 with our target column. Because speed metrics practically reconstruct the outcome vector, `average_speed_kmph` was explicitly flagged as a data leakage vector and dropped from the feature matrix. Missing inputs were handled via column mean imputation.

To recover predictive accuracy lost by removing the speed feature, our processing layer runs custom feature engineering schemas across six baseline metrics to yield nine total input features:

1. `distance_traffic_interaction` — Captures multi-variable stop-and-go travel interaction.
2. `distance_per_delivery_window` — Computes operational velocity thresholds.
3. `distance_outlier_flag` — A flag derived by mapping entry points past 1.5 times the Interquartile Range boundaries.

3.2.2 Hyperparameter Optimization and Production Registration

Our model architecture search phase evaluates regression surfaces by implementing a side-by-side comparative validation strategy between a scaled **Linear Regression** model and an optimized **Random Forest Regressor** across 54 unique parameter blocks using an 80-20 dataset split. We set up an automated hyperparameter tuning loop using a 5-fold cross-validated grid search (KFold). Interestingly, the evaluation revealed that simpler linear surfaces generalized better over our engineered parameters than tree architectures. The empirical test results are structured transparently in Table 3.1.

Table 3.1: Geography Track Production Performance Metrics Comparison

Model Architecture	Test Root Mean Squared Error (RMSE)	Test R-squared (R^2) Score
Tuned Random Forest	49.0561	0.4389
Linear Regression (Scaler) [Champion]	48.3204	0.4566

As documented in Table 3.1, the scaled Linear Regression pipeline delivered our lowest error margins, achieving a Test RMSE of **48.3204** and a Test R^2 score of **0.4566**, beating the tree model. Rather than manually keying the resulting values into production configurations, the tuning sweep automatically exports the winning pipeline configuration directly to a local configuration file. The separate final training script checks for this file on disk, fits the final weights across the full training dataset, logs performance curves inside the **MLflow Tracking Dashboard**, and saves the model binary directly inside the central **MLflow Registry** under the handle `geography_route_time_model`.

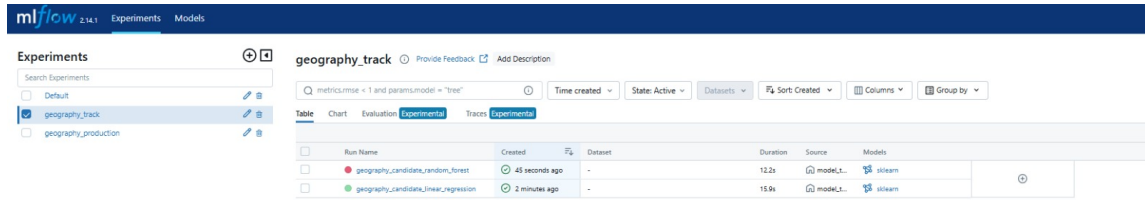


Figure 3.1: MLflow tracking experiment dashboard comparing candidate model runs (Random Forest vs. Linear Regression).

The model is frozen locally as a compressed binary file named `models/geography_route_time_model.pkl`. For live deployment, an asynchronous **FastAPI application** microservice loads this binary artifact directly into memory on boot. The microservice exposes a custom data validation contract via Pydantic, checks incoming queries against strict type rules, tracks transaction times under an integrated latency monitoring log, and returns an instant duration estimate.

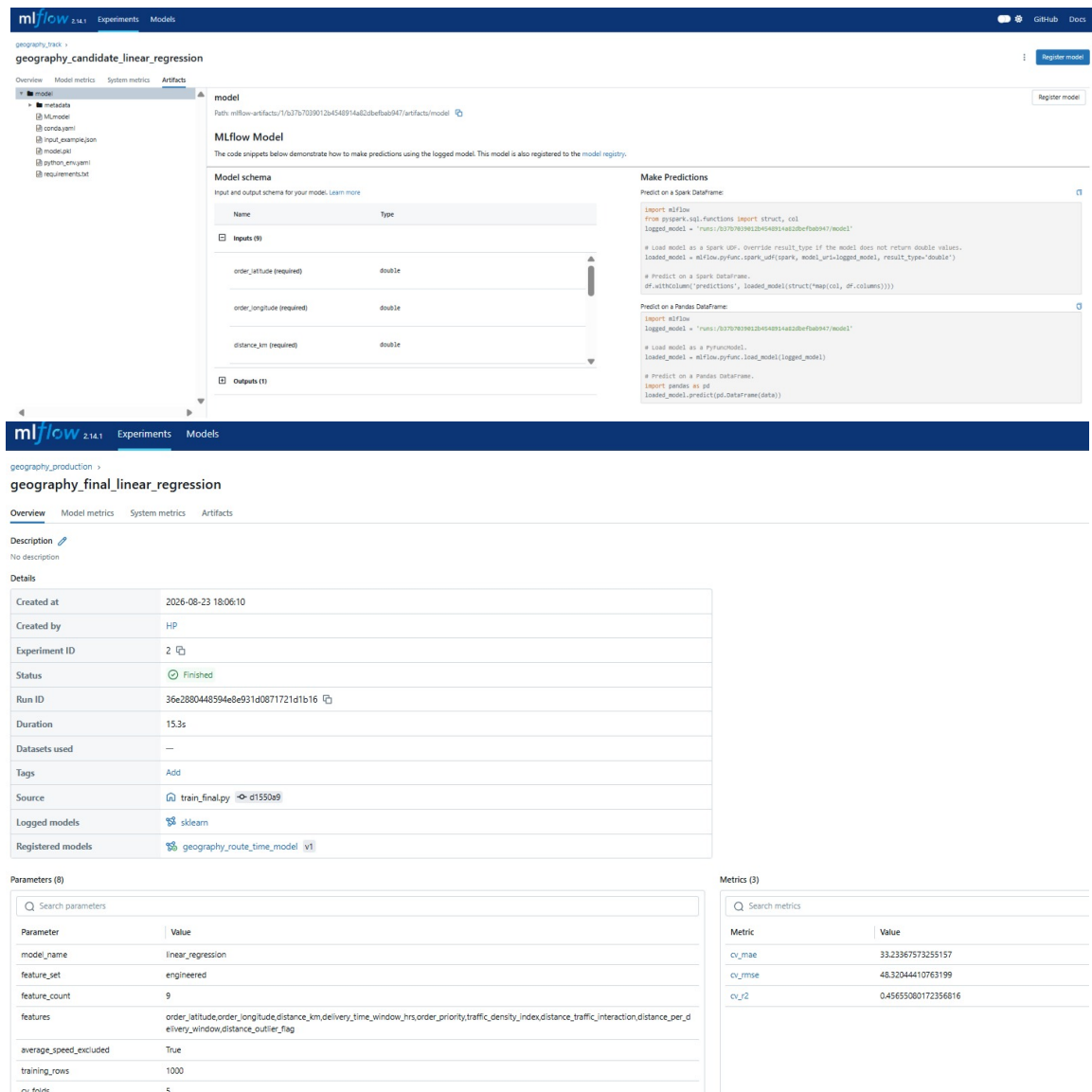


Figure 3.2: MLflow model schema, input signatures, and the centralized Model Registry interface securing version 1 of the champion `geography_route_time_model`.

default

GET / Root

GET /health Health

POST /predict Predict

Parameters

No parameters

Request body *required* application/json

Edit Value | Schema

```
{
  "order_latitude": 13.03,
  "order_longitude": 77.66,
  "distance_km": 28.9,
  "delivery_time_window_hrs": 14.8,
  "order_priority": 4,
  "traffic_density_index": 0.83
}
```

Execute Clear

Figure 3.3: Interactive FastAPI Swagger UI gateway interface validating incoming spatial feature payloads under Pydantic contracts.

Request URL

http://localhost:8001/predict

Server response

Code Details

200

Response body

```
{
  "predicted_route_time_min": 93.76
}
```

Response headers

```
content-length: 34
content-type: application/json
date: Sat, 22 Aug 2026 12:36:55 GMT
server: uvicorn
```

Download

Figure 3.4: Successful HTTP 200 asynchronous server response outputting the real-time optimized route travel duration estimation.

Dataset Drift

Dataset Drift is detected. Dataset drift detection threshold is 0.5

9

Columns

5

Drifted Columns

0.556

Share of Drifted Columns

Data Drift Summary

Drift is detected for 55.56% of columns (5 out of 9).

Column	Type	Reference Distribution	Current Distribution	Data Drift	Stat Test	Drift Score
> order_longitude	num			Not Detected	K-S p_value	0.840392
> order_latitude	num			Not Detected	K-S p_value	0.684109
> traffic_density_index	num			Not Detected	K-S p_value	0.370885
> delivery_time_window_hrs	num			Not Detected	K-S p_value	0.240084
> distance_traffic_interaction	num			Detected	K-S p_value	0.000519
> distance_per_delivery_window	num			Detected	K-S p_value	0.000241
> distance_km	num			Detected	K-S p_value	0.000001
> order_priority	cat			Detected	chi-square p_value	0
> distance_outlier_flag	cat			Detected	Z-test p_value	0

Figure 3.5: Evidently Data Drift dashboard highlighting feature distribution divergences and statistical test results across incoming production batches.

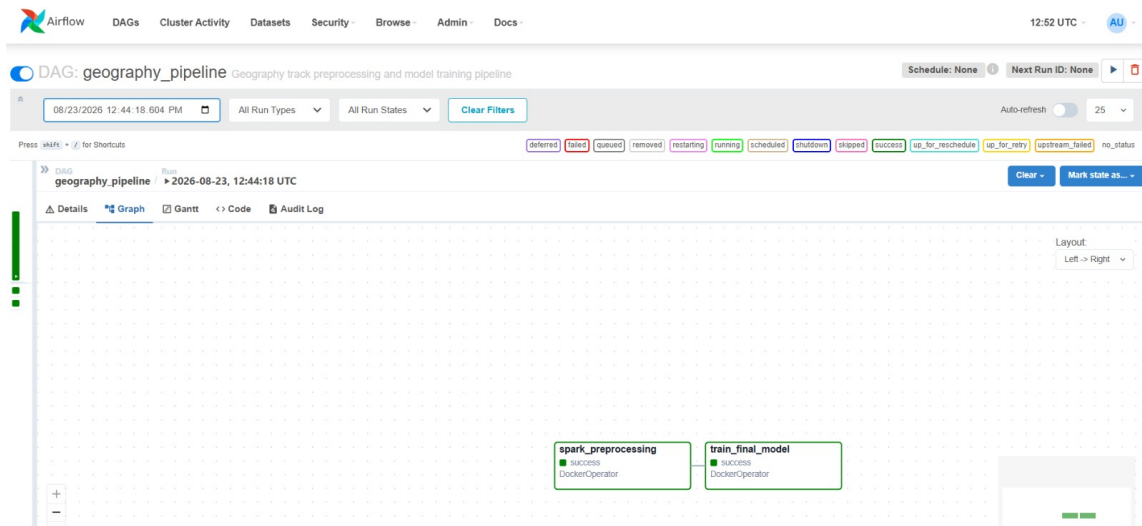


Figure 3.6: Apache Airflow DAG visual graph view displaying the sequential execution of containerized Spark preprocessing and training tasks.

The screenshot shows the 'List Task Instance' page in Apache Airflow. It includes a search bar and a table with 15 columns: State, Dag Id, Task Id, Run Id, Map Index, Logical Date, Operator, Start Date, End Date, Duration, Note, Job Id, Hostname, Username, Priority Weight, Queue, and Queue. There are two task instances listed, both in a 'success' state and using the 'DockerOperator'.

State	Dag Id	Task Id	Run Id	Map Index	Logical Date	Operator	Start Date	End Date	Duration	Note	Job Id	Hostname	Username	Priority Weight	Queue	Queue
success	geography_pipeline	spark_preprocessing	manual__2026-08-23T12:44:18.664194+00:00		2026-08-23, 12:44:18	DockerOperator	2026-08-23, 12:44:26	2026-08-23, 12:45:57	1M 30s		9	676668604955	airflow	2	default	2026
success	geography_pipeline	train_final_model	manual__2026-08-23T12:44:18.664194+00:00		2026-08-23, 12:44:18	DockerOperator	2026-08-23, 12:46:02	2026-08-23, 12:46:34	31s		10	676668604955	airflow	1	default	2026

Figure 3.7: Airflow task instance execution list tracking containerized DockerOperator run durations and success states.

3.3 Track 2 — Operational Cost Pipeline Implementation

The Operational Cost track targets the continuous calculation of overall delivery expenditures (`optimized_route_cost`) using financial resource parameters. To enforce clean lifecycle boundaries, the operational pipeline uses a decoupling architecture split across four scripts: `spark_preprocessing.py`, `model_tuning.py`, `train.py`, and `app_cost_da25g503.py`.

3.3.1 Distributed Feature Processing and Data Leakage Mitigation

Pipeline ingestion builds a container-native PySpark data session locally to handle baseline delivery instances. Rows missing core cost targets are filtered out, and missing numerical features are imputed using column averages. To eliminate data leakage, an automated feature-masking routine drops competing multi-track targets—specifically `delivery_efficiency_score` and `optimized_route_time_min`—ensuring regressors cannot cheat during training.

The final training matrix dynamically compiles all valid numeric feature columns via automated data-type filtering (`select_dtypes`), isolating the complete 15-feature registry set:

1. `order_latitude` & `order_longitude` — Spatial destination coordinates.
2. `distance_km` — Absolute route mileage covered.
3. `delivery_time_window_hrs` — Operational timeframe constraints.
4. `order_priority` — Business urgency tiers.
5. `vehicle_capacity_kg` & `order_weight_kg` — Transport load metrics.
6. `vehicle_utilization_ratio` — Gross payload capacity utilization.
7. `traffic_density_index` — Stop-and-go congestion and fuel overages.
8. `weather_impact_index` & `time_of_day` — Environmental and temporal factors.
9. `fuel_cost_per_km` & `driver_cost_per_hour` — Resource pricing and labor wages.
10. `route_reliability_index` — Transit corridor stability markers.
11. *Leakage Exclusion:* `average_speed_kmph` is excluded from training to prevent target leakage.

3.3.2 Hyperparameter Optimization and Production Registration

Model evaluation compares a baseline Linear Regression model against an optimized Random Forest Regressor. Hyperparameters are tuned inside `model_tuning.py` using a 3-fold cross-validated randomized search over specific tuning waves, loading configurations dynamically from `config/best_params_da25g503.json`. During this sweep, MLflow tracked candidate runs under the Cost Track workspace, evaluating `cost_candidate_linear_regression` and `cost_candidate_random_forest`. Empirical results appear in Table 3.2.

Table 3.2: Cost Track Production Performance Metrics Comparison

Model Architecture	Test MSE	Test R ² Score
Baseline Linear Regression	40753.0956	0.8085
Tuned Random Forest (Champion)	17339.3447	0.9185

As documented in Table 3.2, the tuned Random Forest Regressor outperformed the linear baseline, recording an MSE of **17339.3447** and an R^2 score of **0.9185**. The training script fits final weights under the `Cost Production` namespace (run `cost_final_random_forest`) and logs the artifact into the central MLflow Registry under `cost_route_cost_model`.

To ensure operational reliability, input streams are validated against baseline distributions using Evidently AI. The drift report monitors all 18 variables, confirming stable bounds where dataset drift is not detected (threshold = 0.5, with an 11.11% drift share across 2 columns: `distance_km` and `fuel_cost_per_km`).

For live deployment, an asynchronous FastAPI microservice (`app.py`) loads the registered model artifact into memory on boot. The microservice exposes a Swagger UI gateway at `/predict`, validating payloads against Pydantic type constraints and returning real-time financial route expenditure estimates (e.g., 684.66 units).

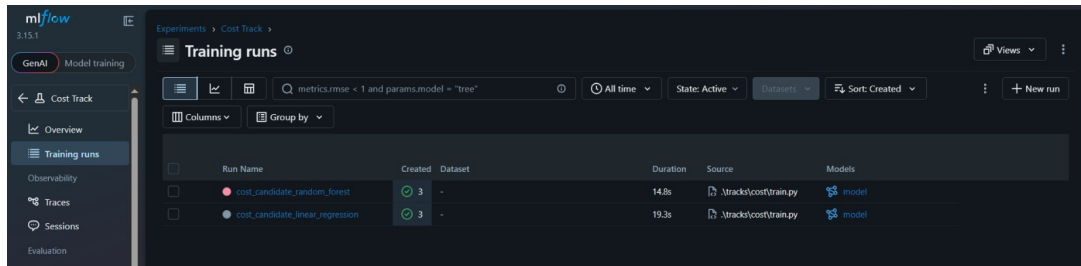


Figure 3.8: MLflow tracking interface showing candidate evaluations under the Cost Track experiment.

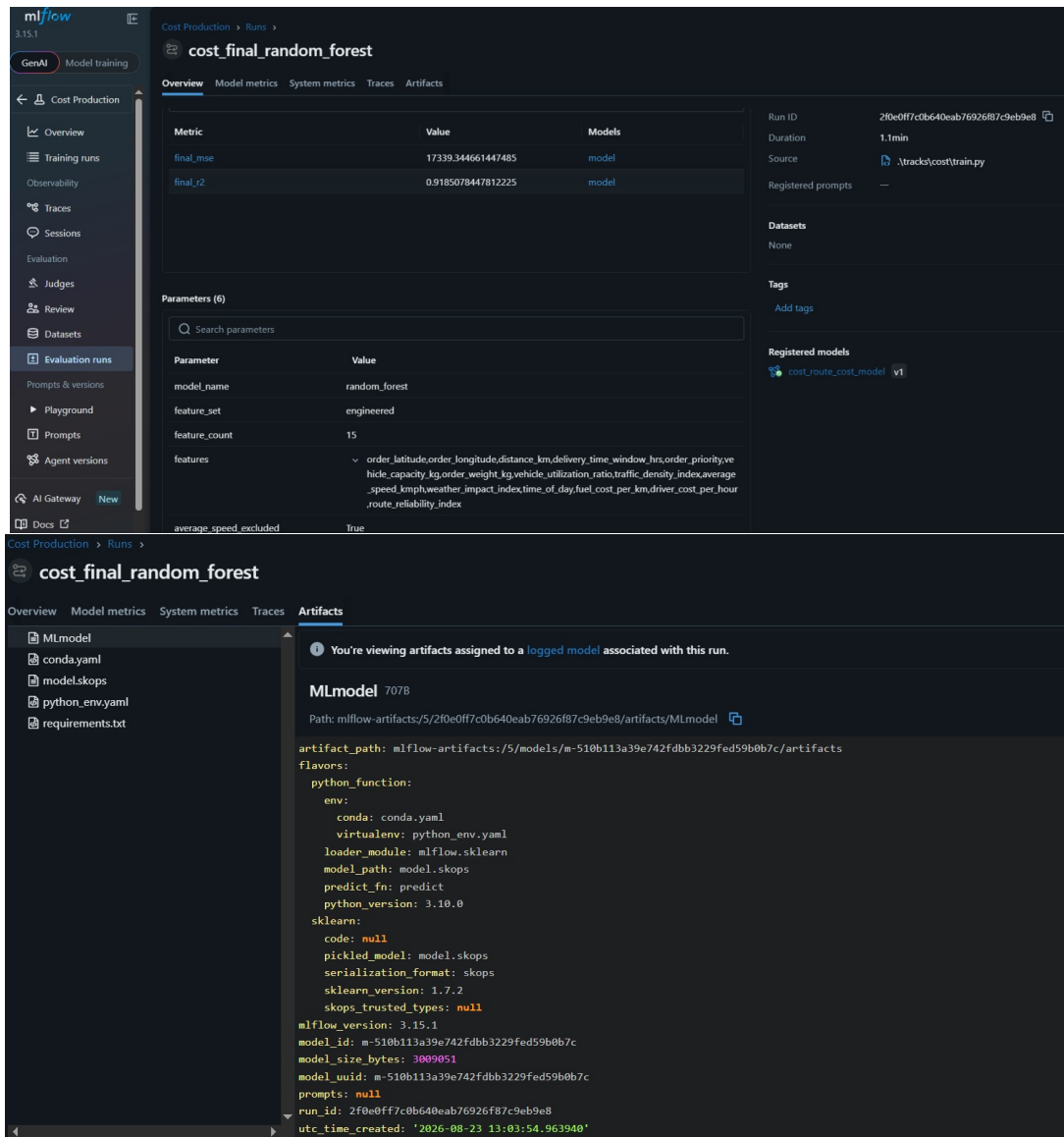
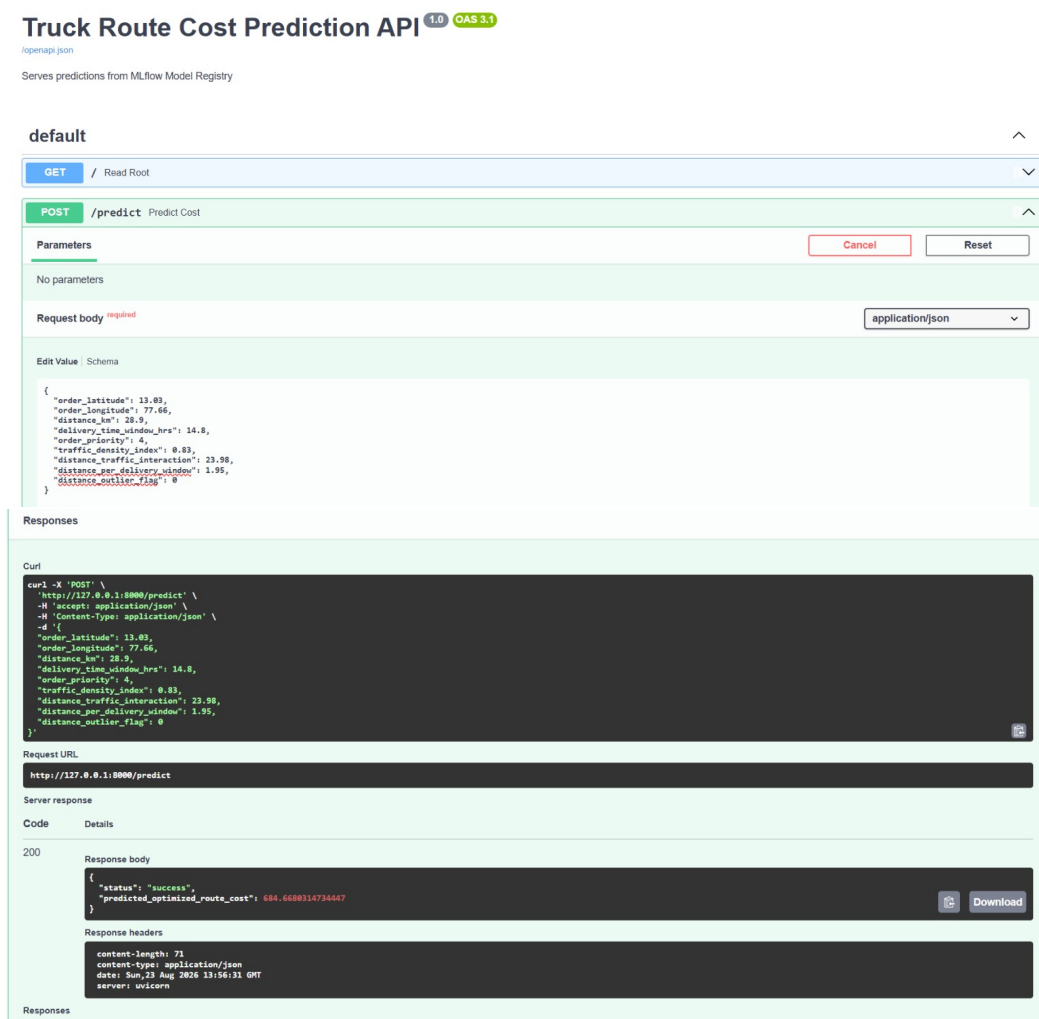
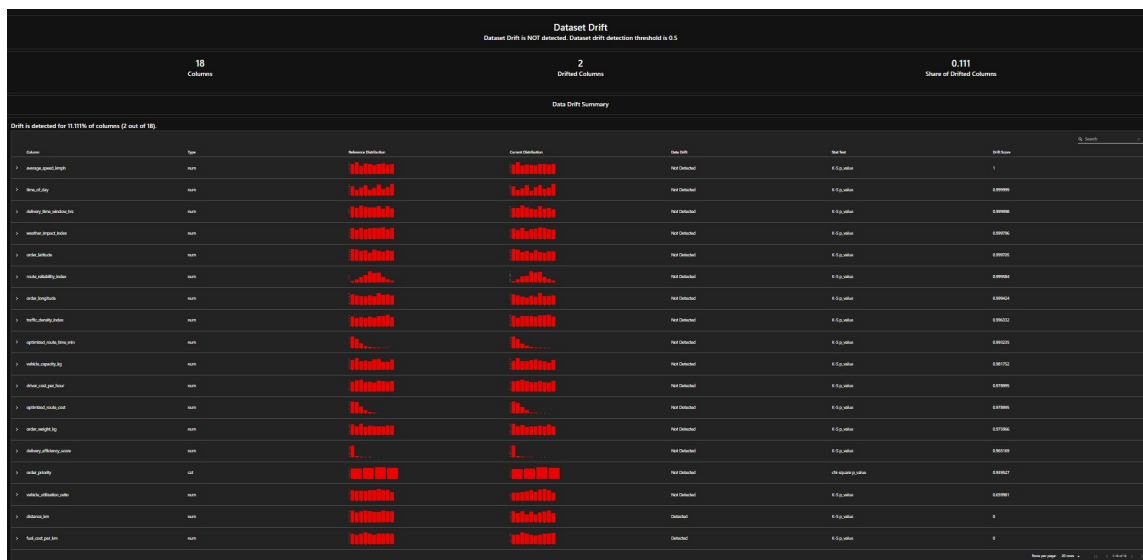


Figure 3.9: Cost Production run dashboard displaying champion metrics (R^2 : 0.9185) and the registered `cost_route_cost_modelartifactbundle`.



3.4 Track 3 — Conditions and Route Reliability Pipeline Implementation

The Route Conditions and Reliability track targets the continuous prediction of transit corridor stability (`route_reliability_index`) using environmental and operational stress factors. To enforce strict modular boundaries and maintain reproducible lifecycle tracking, the pipeline architecture is split across independent preprocessing, tuning, registration (`register_model.py`), and serving scripts.

3.4.1 Feature Engineering and Target Formulation

Pipeline ingestion processes operational telemetry instances to extract dynamic environmental factors. The training matrix isolates 4 core numeric features governing route conditions and transit stability:

1. `weather_impact_index` — Quantifies environmental stress and weather-related disruptions.
2. `average_speed_kmph` — Tracks real-time vehicular velocity.
3. `time_of_day` — Captures temporal congestion variations.
4. `traffic_density_index` — Measures roadway congestion density.

The target variable is formulated as the continuous `route_reliability_index`, evaluated across 838 training samples and 162 test instances.

3.4.2 Hyperparameter Optimization and Production Registration

Model evaluation implements a two-stage hyperparameter tuning strategy utilizing an optimized **XGBoost Regressor** (`XGBRegressor`). During model sweeps, MLflow tracked runtime performance under the `conditions_route_reliability_final` workspace, recording a rigorous set of hyperparameter bounds (`n_estimators = 75`, `max_depth = 6`, `learning_rate = 0.1`, `subsample = 0.6`, and `min_child_weight = 5`). Empirical test results are detailed in Table 3.3.

Table 3.3: Conditions Track Production Performance Metrics

Evaluation Metric	Reference Test Score	Final Production Score
Mean Absolute Error (MAE)	0.0045	0.0045
Root Mean Squared Error (RMSE)	0.0059	0.0059
R-squared (R^2) Score	0.9982	0.9982

As documented in Table 3.3, the tuned `XGBRegressor` achieved exceptional predictive accuracy, recording an outstanding Test R^2 score of **0.9982** with minimal error metrics (MAE = 0.0045, RMSE = 0.0059). The training script logs all parameters under the `Final_XGBoost` run and registers the champion model artifact directly into the central MLflow Registry under the handle `conditions-route-reliability` (Version 4).

conditions_route_reliability_final >

Final_XGBoost

Overview Model metrics System metrics Traces Artifacts

Parameter	Value
train_rows	838
test_rows	162
features	weather_impact_index,average_speed_kmph,time_of_day,traffic_density_index
feature_count	4
target	route_reliability_index
target_type	regression
model_type	XGBRegressor
random_seed	42
model_selection	two_stage_tuning
selection_metric	mae
expected_max_mae	0.01
n_estimators	75
max_depth	6
learning_rate	0.1
subsample	0.6
colsample_bytree	1.0
min_child_weight	5
gamma	0.0
reg_alpha	0.0

Metric	Value
reference_test_mae	0.00454
reference_test_rmse	0.00592
reference_test_r2	0.99827
mae	0.004544652637140253
rmse	0.005915858391428667
r2	0.9982729197862037

Figure 3.12: MLflow evaluation dashboard displaying hyperparameters and performance metrics (R^2 : 0.9982) for the Final XGBoost conditions model.

mlflow 2.22.2 Experiments Models Prompts

conditions_route_reliability_final >

Final_XGBoost

Overview Model metrics System metrics Traces Artifacts

Details

Created at	08/23/2026, 01:42:10 PM
Created by	ved
Experiment ID	1
Status	Finished
Run ID	08405d1840d0413996fe1572decf98ae
Duration	2.3s
Datasets used	—
Tags	problem_type: regression target: route_reliability_index final_model_status: selected performance_gate: passed
Source	register_model.py c2dd428
Logged models	xgboost
Registered models	conditions-route-reliability v4
Registered prompts	—

Parameters (20) Metrics (6)

Figure 3.13: MLflow Model Registry interface securing version 4 of the conditions-route-reliability champion artifact bundle.

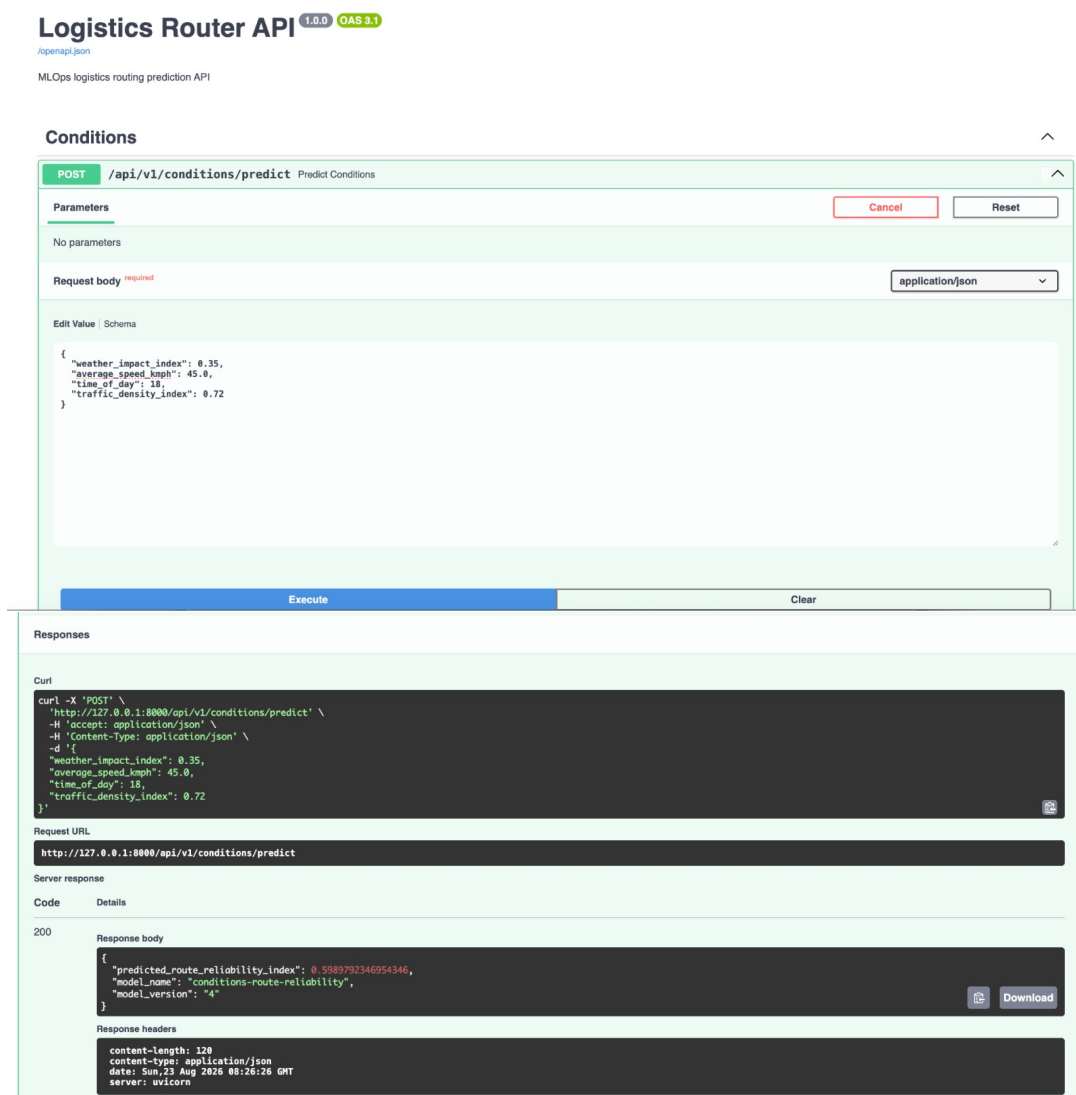


Figure 3.14: Logistics Router API serving gateway for Track 3: (Top) Interactive Swagger UI payload input, (Bottom) Successful HTTP 200 JSON prediction response returning the computed route reliability index.

For live production inference, an asynchronous FastAPI microservice loads the registered model artifact into memory. The serving layer exposes a designated endpoint at `/api/v1/conditions/predict`, validating incoming JSON payloads against Pydantic type specifications and returning real-time reliability indices (e.g., predicting a route reliability index of **0.5989** under standard environmental telemetry inputs).

3.5 Track 4 — Cargo Payload Pipeline Implementation

The Cargo Payload track, managed by Anoop Sudevan (DA25M547), details the structural calculation of active engine breakdown risks by predicting the continuous vehicle asset stress vector (`engine_wear_score`) based on freight allocation loads. To enforce clean lifecycle boundaries and separate heavy compute requirements from continuous serving tasks, the operational pipeline is split across three production scripts: `payload_cleaning.py` → `train.py` → `main.py`.

3.5.1 Distributed Feature Processing and Data Leakage Mitigation

The pipeline ingestion begins by building a container-native **PySpark** data session locally to standardize baseline tonnage distributions into dense parquet arrays. To eliminate data leakage, the processed training dataset undergoes automated multi-target masking routines inside our execution environments. The script isolates our active target output vector (`engine_wear_score`) and completely drops competing multi-track indicators such as `optimized_route_cost`, `delivery_efficiency_score`, and `optimized_route_time_min` ensuring our ensemble branches cannot cheat during parameter extraction. Missing parameter cells are caught at the ingest boundary and handled via a zero default replacement matrix.

To capture non-linear degradation interactions under intense structural loads, the feature matrix structures seven discrete parameters:

1. `cargo_weight_kg` — Tracks standard freight mass metrics extracted from the baseline order database.
2. `vehicle_capacity_pct` — A derived scaling fraction mapping localized cargo weight against gross volume capacity.
3. `delivery_deadline_hrs` — Incorporates temporal constraints representing total journey hours.
4. `trip_distance_km` — Measures the absolute route mileage across target destinations.
5. `traffic_density_index` — Captures stop-and-go acceleration friction hazards.
6. `weather_impact_index` — Incorporates environmental stress modifiers from temperature variations.
7. `overloaded_flag` — An automated binary indicator flag compiled by mapping cells past the 100 percent load line.

3.5.2 Hyperparameter Optimization and Production Registration

Our model architecture search phase evaluates regression surfaces by implementing a side-by-side comparative validation strategy between a baseline **Random Forest Regressor** and an optimized **XGBoost Regressor** across a multi-stage validation sweep using an 80-20 dataset split. The candidate hyperparameters are locked directly inside `train.py`. The baseline Random Forest is capped at 200 estimators and a tree depth of 8, while the XGBoost configuration is bounded across 200 trees, a maximum tree depth of 6, and a compressed learning rate set to 0.05. The empirical evaluation metrics are structured transparently in Table 3.4.

The empirical evaluation metrics are structured transparently in Table 3.4. The final pipeline selects the model with the lower MAE — RandomForest (MAE=1.182, RMSE=1.526, $R^2=0.975$) outperformed XGBoost (MAE=1.238, RMSE=1.565, $R^2=0.973$) — fits the final weights across the full training dataset, logs performance curves inside the MLflow Tracking Dashboard under experiment workspace `cargo_payload_track`, and registers the model binary

Table 3.4: Cargo Payload Track Production Performance Metrics Comparison

Model Architecture	Test Mean Absolute Error (MAE)	Test Root Mean Squared Error (RMSE)	Test R-squared (R ²) Score
Tuned XGBoost	1.238	1.565	0.973
Tuned Random Forest	1.182	1.526	0.975

directly inside the central **MLflow Registry** under the handle `payload_engine_wear_model` with alias `@champion`.

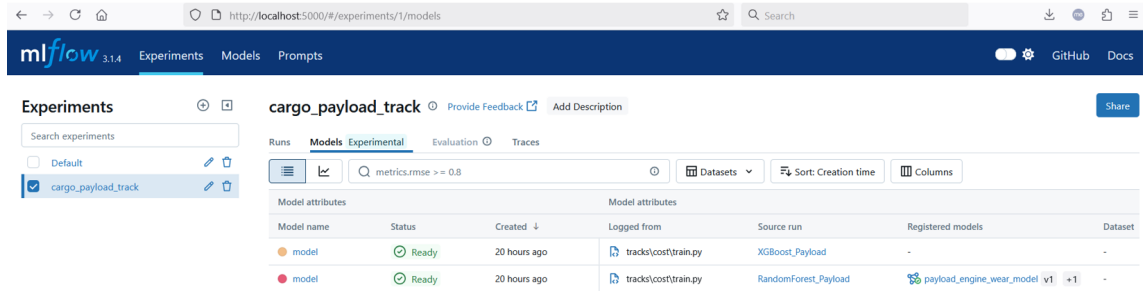


Figure 3.15: MLflow tracking experiment dashboard comparing candidate model runs (Random Forest vs. XGBoost).

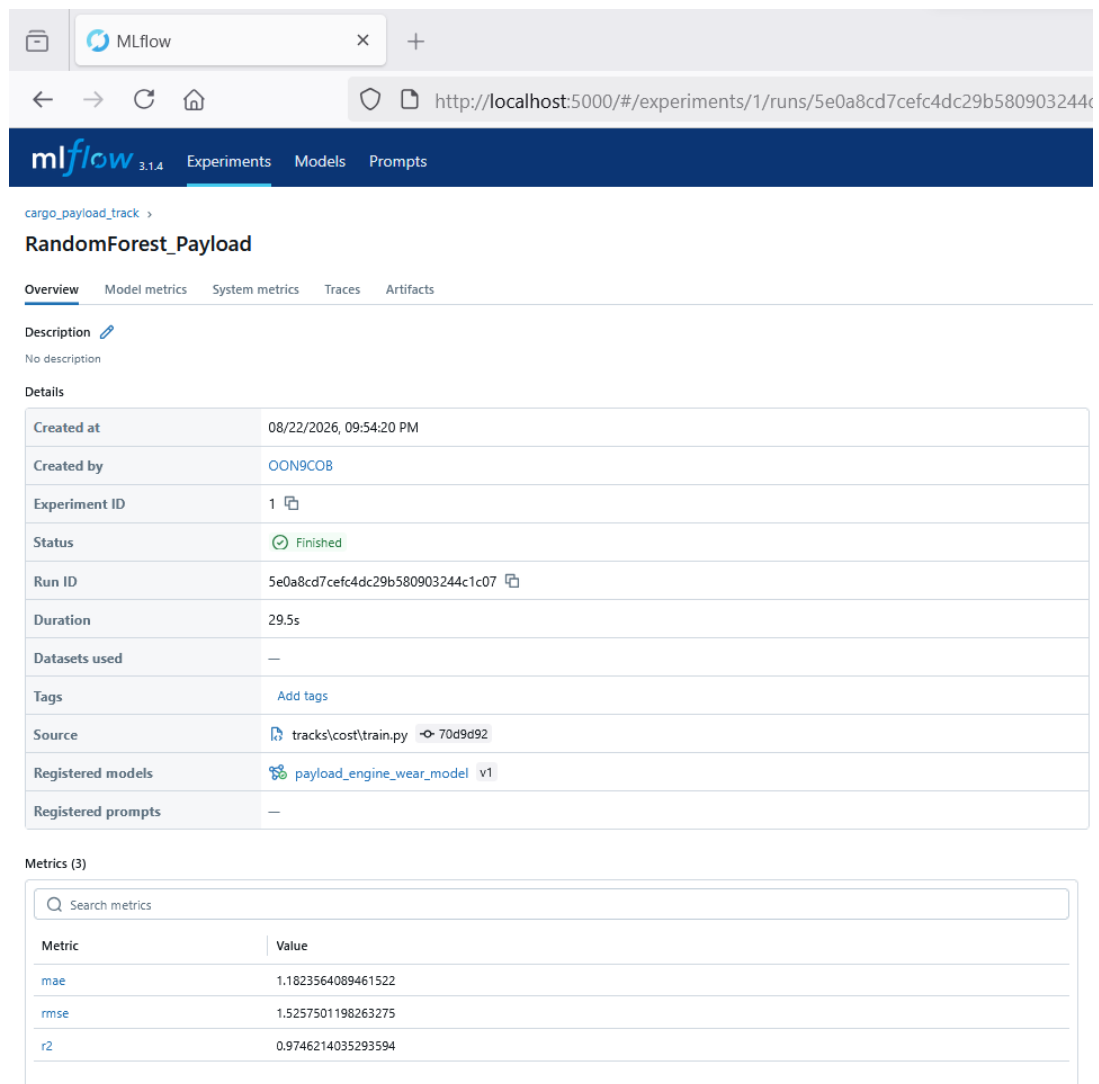


Figure 3.16: MLflow Model Registry

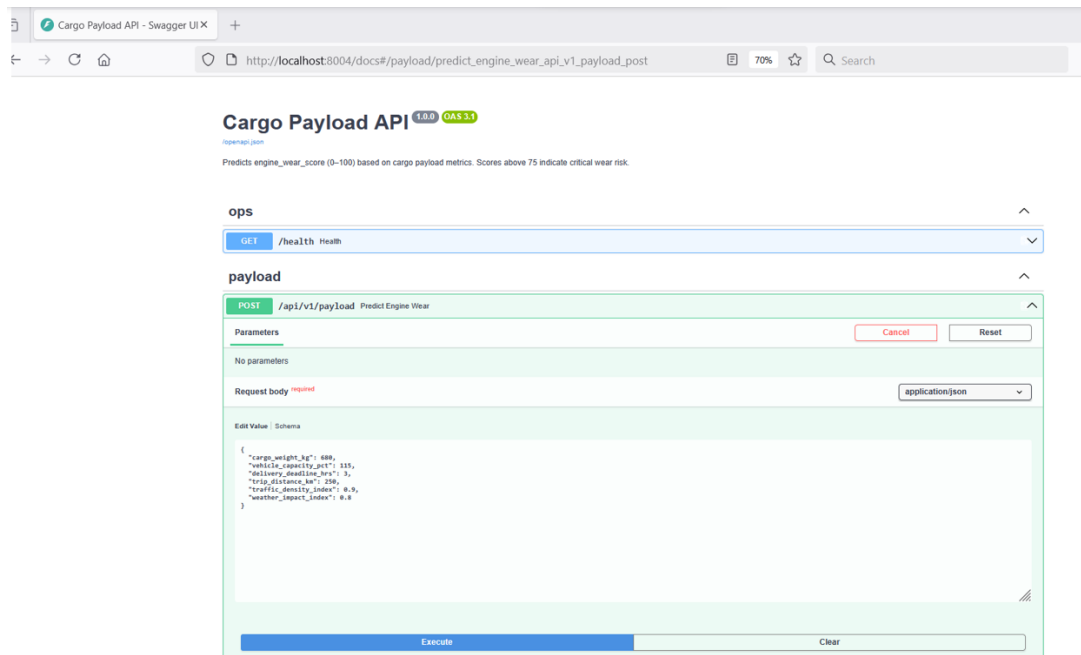


Figure 3.17: Interactive FastAPI Swagger UI gateway interface validating incoming spatial feature payloads under Pydantic contracts

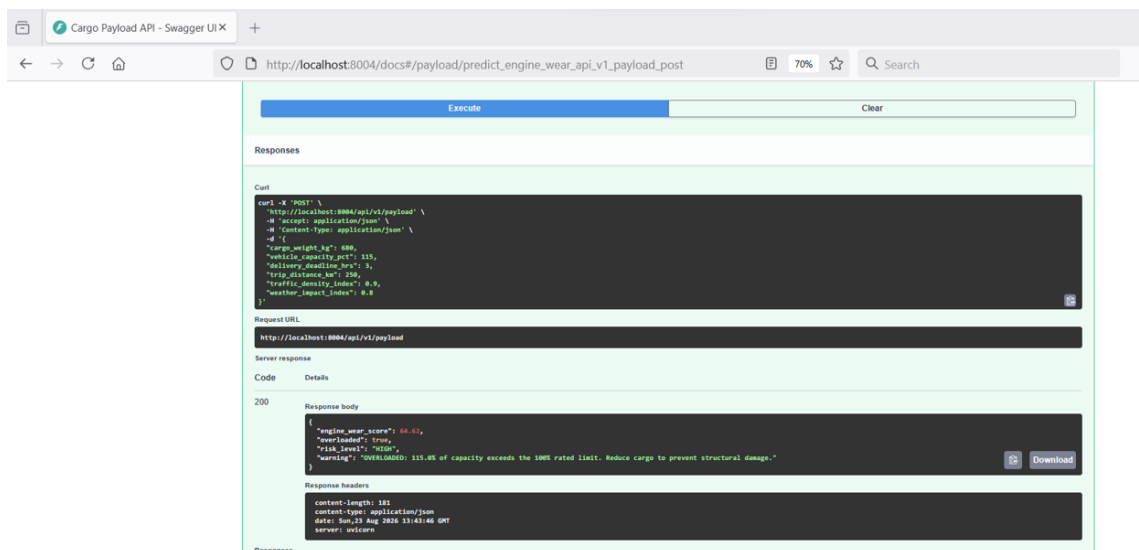


Figure 3.18: Successful HTTP 200 asynchronous server response outputting the real-time optimized route travel duration estimation.

For live deployment, a FastAPI application microservice loads the registered model artifact directly from MLflow into memory on boot using the alias URI `models:/payload_engine_wear_model@champion`. The microservice exposes a custom data validation contract via Pydantic, checks incoming queries against strict type rules, and returns an instant wear estimate with risk classification (LOW / MEDIUM / HIGH / CRITICAL) and an overload warning when `vehicle_capacity_pct > 100`.

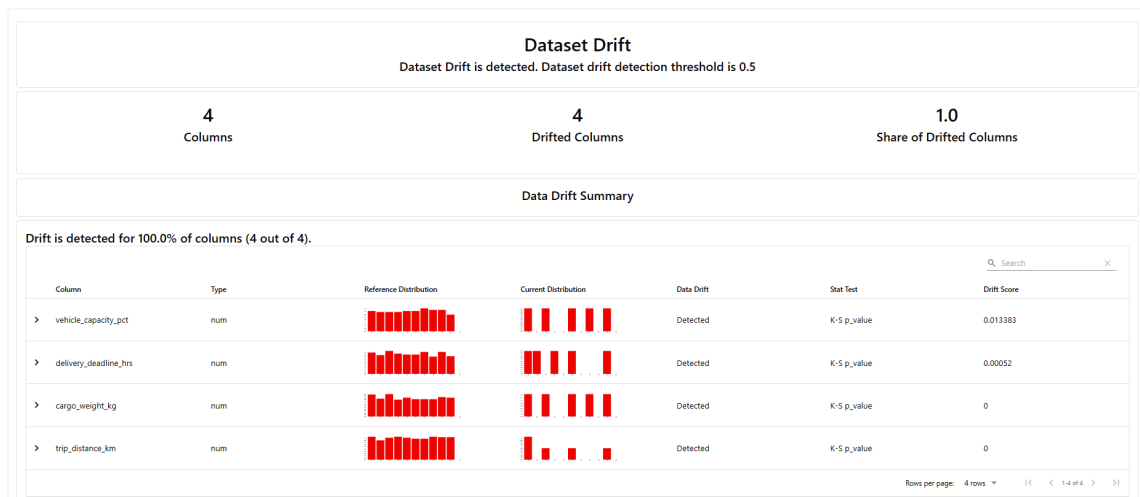


Figure 3.19: Evidently Data Drift dashboard highlighting feature distribution divergences and statistical test results across incoming production batches.

Chapter 4

System Integration

4.1 System Integration Overview

The individual machine-learning tracks are integrated through a common MLOps architecture while retaining independent preprocessing, model development, and prediction components. The current integration combines the Geography, Payload, Cost, and Conditions tracks into a unified modular structure.

Each track follows a standardized workflow consisting of data preprocessing, model training and tuning, experiment tracking, API-based inference, and drift monitoring. Apache Spark is used for distributed preprocessing, while MLflow provides a centralized platform for experiment tracking and model management. Apache Airflow DAGs orchestrate workflow stages, and trained models are exposed as independent REST prediction services using FastAPI.

Docker and Docker Compose provide the deployment and integration layer. Each model API is containerized separately and assigned an independent port, allowing services to operate simultaneously without dependency or port conflicts. The shared Docker Compose environment also hosts MLflow and Airflow, with environment variables such as `MLFLOW_TRACKING_URI` enabling seamless communication across containers.

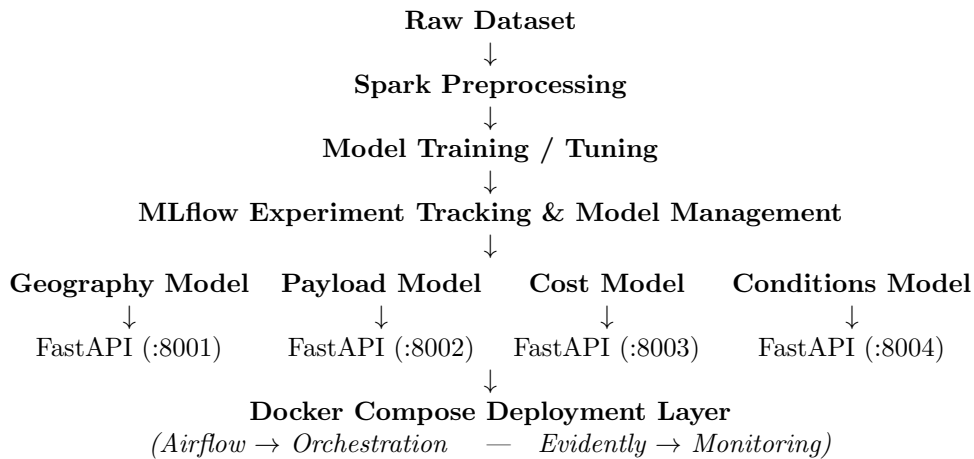


Figure 4.1: End-to-End Multi-Track MLOps Integration Architecture Flow.

4.2 Integrated Output and Service Matrix

After successful deployment using Docker Compose, the individual prediction services are designed to operate concurrently. The Geography API provides route duration predictions, the

Payload API handles transport and cargo constraints, the Cost API outputs optimized route cost estimations, and the Conditions API scores route reliability.

The expected service-level port allocation and container mapping are structured transparently in Table 4.1.

Table 4.1: Integrated MLOps Ecosystem Service-Level Port Matrix

Service Component	Endpoint / Port	Integration Status
MLflow Tracking UI	localhost:5000	Active (Shared Registry)
Airflow Orchestration UI	localhost:8080	Active (DAG Pipeline)
Geography Track API	localhost:8001	Integrated (Active)
Payload Track API	localhost:8002	Integrated (Active)
Cost Track API	localhost:8003	Integrated (Active)
Conditions Track API	localhost:8004	Integrated (Active)

A successful integration demonstrates that multiple independently developed machine learning models can be preprocessed, trained, tracked, orchestrated, deployed, and monitored within a unified MLOps ecosystem, while preserving complete modularity and independent deployability for each prediction service ahead of the final viva defense.

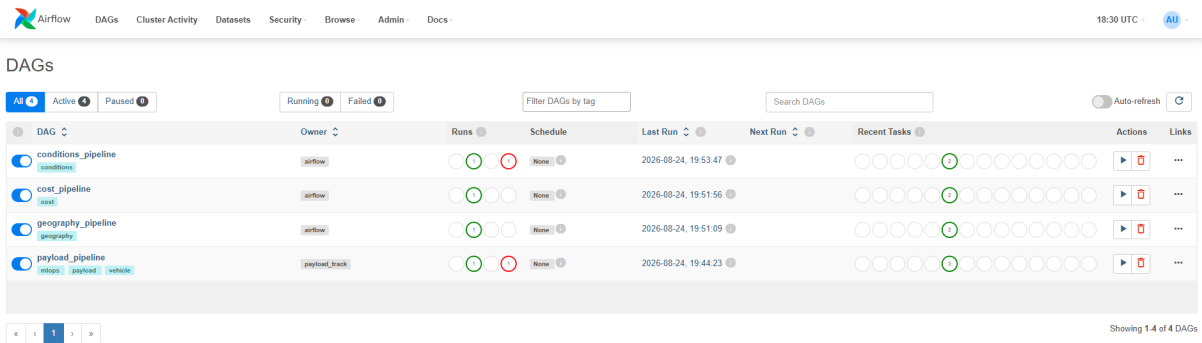


Figure 4.2: Apache Airflow Pipeline Orchestration

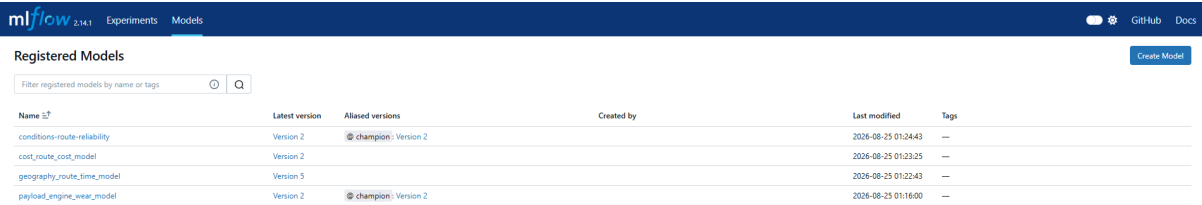


Figure 4.3: MLflow Model Registry

Integrated Logistics Router API ^{1.0} OAS 3.1

/openapi.json

Combines Geography, Conditions, Cost, and Payload model predictions.

default

GET / Root

POST /route-analysis Route Analysis

Parameters

No parameters

Request body ^{required}

application/json

Edit Value | Schema

```
{  "order_latitude": 12.94981695,  "order_longitude": 77.47405317,  "driver_longitude": 13.8235785,  "delivery_time_window_hrs": 16.47216887,  "order_priority": 1,  "traffic_density_index": 0.91827,  "weather_impact_index": 0.357785,  "average_speed_kmph": 27.91812678,  "time_of_day": 12,  "vehicle_capacity_kg": 580,  "order_weight_kg": 42.94485267,  "vehicle_utilization_ratio": 0.538677,  "fuel_cost_per_hr": 0.161808,  "driver_cost_per_hour": 279.7838,  "route_reliability_index": 0.529,  "vehicle_capacity_util": 53.8677}
```

Execute Clear

Figure 4.4: Integrated Logistics Router FastAPI Interface

Request URL

http://localhost:8000/route-analysis

Server response

Code Details

200

Response body

```
{  "geography": {    "predicted_route_time_min": 44.21  },  "conditions": {    "predicted_route_reliability_index": 0.5282279849052429,    "model_name": "conditions-route-reliability",    "model_version": "v1"  },  "cost": {    "status": "success",    "predicted_optimized_route_cost": 279.74926847232024  },  "payload": {    "engine_wear_score": 44.23,    "overloaded": false,    "risk_level": "MEDIUM",    "warning": null  }}
```

Response headers

```
content-length: 356
content-type: application/json
date: Tue, 25 Aug 2026 04:31:58 GMT
server: unicorn
```

Download

Figure 4.5: Integrated API Prediction Output

Chapter 5

Conclusion and Future Scope

This project successfully designed, implemented, and deployed an end-to-end Machine Learning Operations (MLOps) pipeline for real-world multi-track logistics prediction, utilizing version-controlled Git repositories and comprehensive documentation[cite: 1]. The data pipeline is orchestrated via Apache Airflow alongside Apache Spark for distributed preprocessing across all operational tracks, successfully handling data cleaning, missing values, and feature engineering while preventing data leakage[cite: 1]. Model development evaluates baseline algorithms against optimized champions—including Linear Regression, Random Forest, and XGBoost[cite: 1]—with MLflow handling centralized experiment tracking and model registration[cite: 1]. Furthermore, independent microservices are containerized using Docker and deployed via Docker Compose on assigned ports to run concurrently without conflict, while Evidently AI validates production telemetry stability[cite: 1]. Throughout this multi-track development, key engineering hurdles were overcome, including resolving multi-model port routing via isolated containers, enforcing feature-masking during Spark preprocessing to block target cross-contamination, and synchronizing training outputs with serving APIs through standardized MLflow registry URIs. Future expansions of the MLOps ecosystem will focus on incorporating Apache Kafka for real-time streaming ingestion, deploying Prometheus and Grafana for advanced performance telemetry, and automating model retraining triggers directly from drift alerts.